Fiber

> 📚 `fiber` 是核心, react 体系的渲染和更新都要以 fiber 作为数据模型, 如果不能理解 fiber, 也无法深入理解 react.

在 `React15` 及以前， `Reconciler` 采用递归的方式创建虚拟DOM，递归过程是不能中断的。如果组件树的层级很深，递归会占用线程很多时间，造成卡顿。

为了解决这个问题， `React16` 将递归的无法中断的更新重构为异步的可中断更新，由于曾经用于递归的虚拟DOM数据结构已经无法满足需要。于是，全新的 `Fiber` 架构应运而生。

`React Fiber` 可以理解为：

`React` 内部实现的一套状态更新机制。支持任务不同 `优先级` ，可中断与恢复，并且恢复后可以复用之前的 `中间状态` 。

Fiber的含义

`Fiber` 包含三层含义：

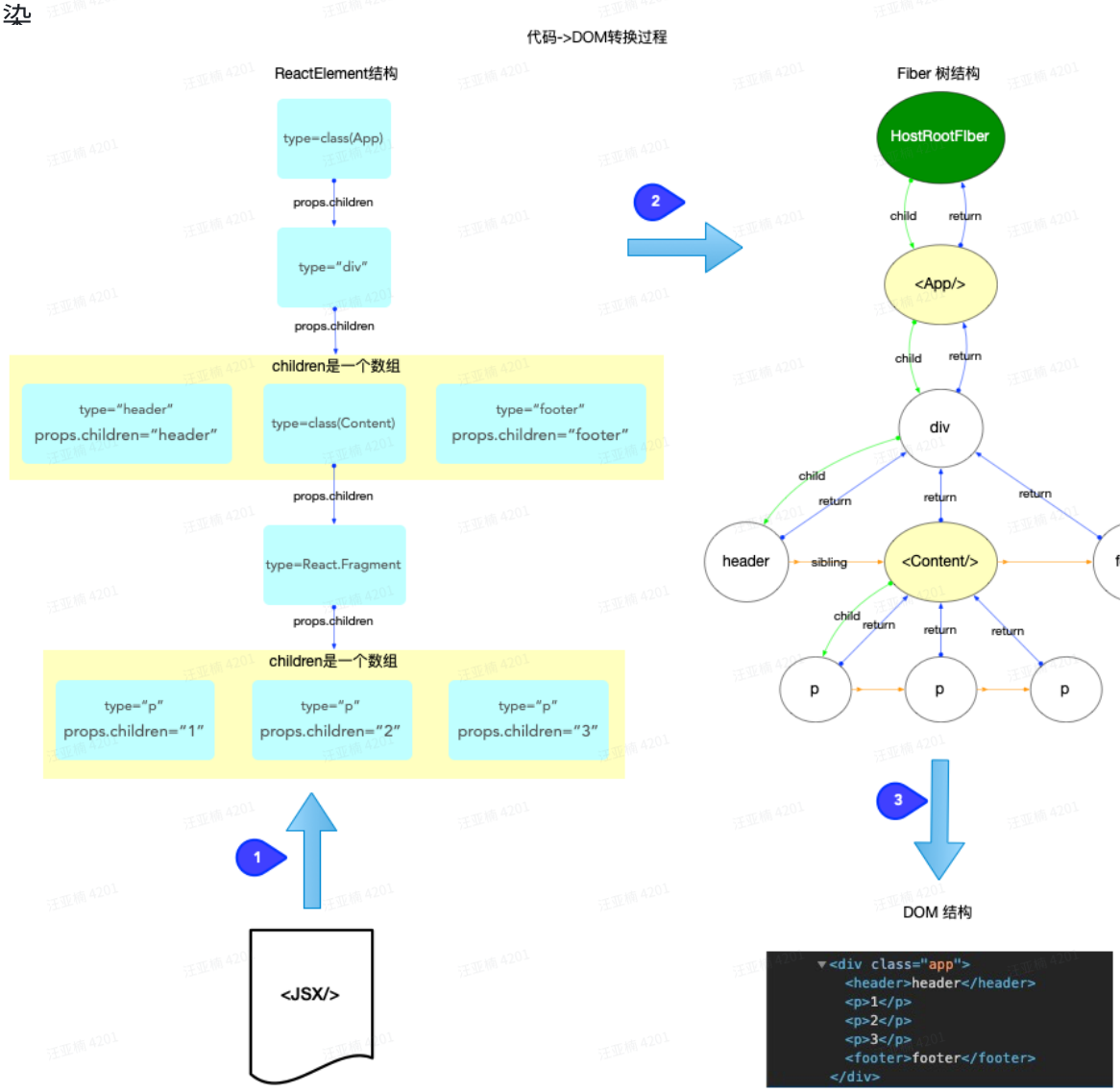
1. 作为架构来说，之前 `React15` 的 `Reconciler` 采用递归的方式执行，数据保存在递归调用栈中，所以被称为 `stack Reconciler` 。 `React16` 的 `Reconciler` 基于 `Fiber节点` 实现，被称为 `Fiber Reconciler` 。
2. 作为静态的数据结构来说，每个 `Fiber节点` 对应一个 `React element` ，保存了该组件的类型（函数组件/类组件/原生组件...）、对应的DOM节点等信息。
3. 作为动态的工作单元来说，每个 `Fiber节点` 保存了本次更新中该组件改变的状态、要执行的工作（需要被删除/被插入页面中/被更新...）。

ReactElement, Fiber, DOM 三者的关系

1. ReactElement 对象(type 定义在shared 包中)
   - 所有采用 `jsx` 语法书写的节点, 都会被编译器转换, 最终会以 `React.createElement(...)` 的方式, 创建出来一个与之对应的 `ReactElement` 对象
2. fiber 对象(type 类型的定义在ReactInternalTypes.js中)
   - `fiber对象` 是通过 `ReactElement` 对象进行创建的, 多个 `fiber对象` 构成了一棵 `fiber树` , `fiber树` 是构造 `DOM树` 的数据模型, `fiber树` 的任何改动, 最后都体现到 `DOM树` .

3. DOM 对象: 文档对象模型

- `DOM` 将文档解析为一个由节点和对象（包含属性和方法的对象）组成的结构集合, 也就是常说的 `DOM树` .
- `JavaScript` 可以访问和操作存储在 DOM 中的内容, 也就是操作 `DOM对象` , 进而触发 UI 渲染

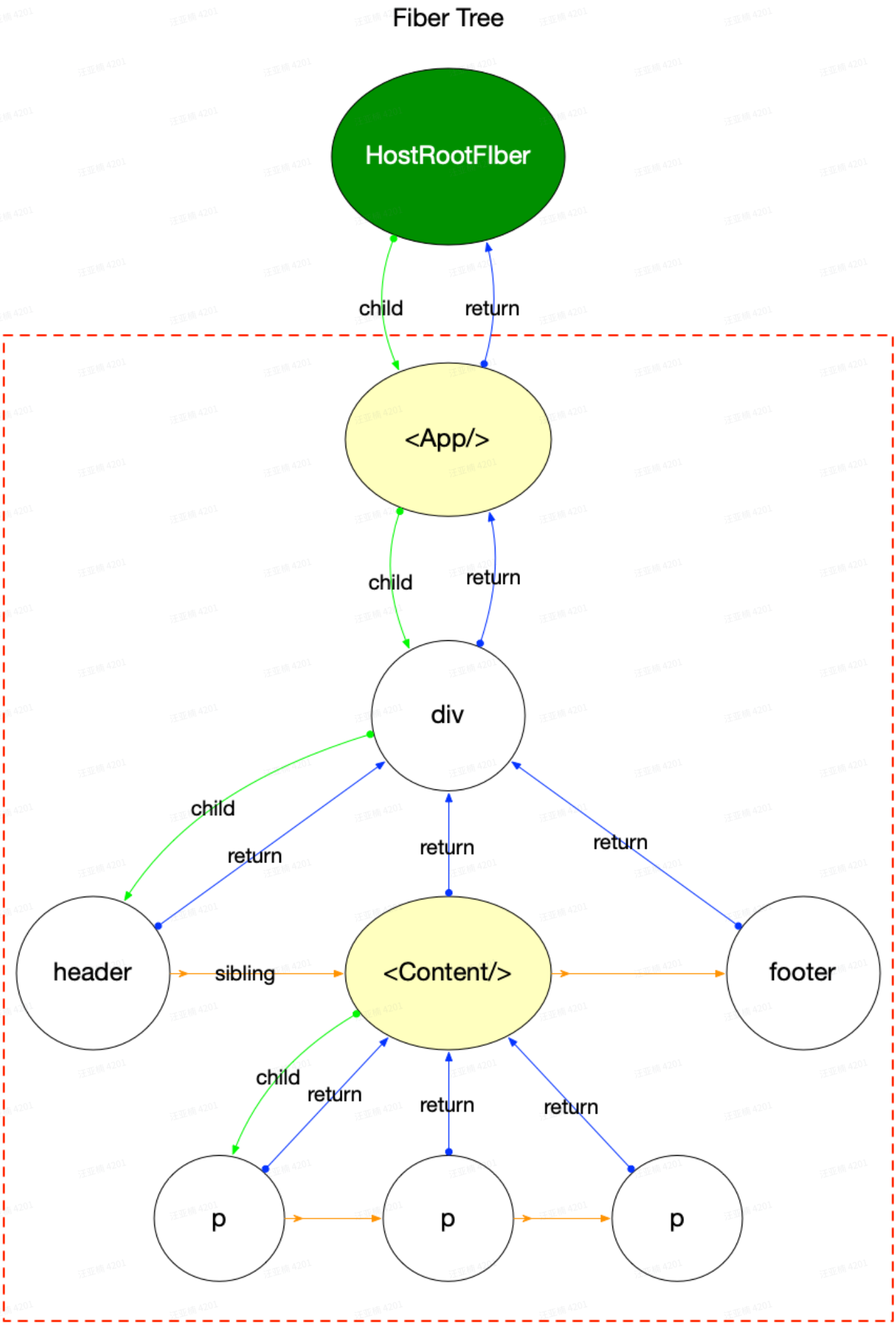


Fiber的定义

```
// 一个Fiber对象代表一个即将渲染或者已经渲染的组件(ReactElement),
// 一个组件可能对应两个fiber(current和WorkInProgress)
export type Fiber = {
  // 作为静态数据结构的属性
  // 表示 fiber 类型, 根据ReactElement组件的 type 进行生成
  tag: WorkTag,
  key: null | string,
  // 一般来讲和ReactElement组件的 type 一致 当用memo的时候不一致
  elementType: any,
  // 一般来讲和fiber.elementType一致.
  // 一些特殊情形下, 比如在开发环境下为了兼容热更新则不一致
```

```
  type: any,
  // 与fiber关联的局部状态节点
  // HostComponent类型指向与fiber节点对应的 dom 节点;
  // 根节点fiber.stateNode指向的是FiberRoot;
  // class 类型节点其stateNode指向的是 class 实例
  // FunctionComponent指向的是什么
  stateNode: any,

  // 用于连接其他Fiber节点形成Fiber树
  return: Fiber | null,
  child: Fiber | null,
  sibling: Fiber | null,
  index: number,
  ref:
    | null
    | (((handle: mixed) => void) & { _stringRef: ?string, ... })
    | RefObject,

  // 作为动态的工作单元的属性
  // 从`ReactElement`对象传入的 props. 用于和`fiber.memoizedProps`比较可以得出属性是否变动
  pendingProps: any,
  // 上一次生成子节点时用到的属性, 生成子节点之后保持在内存中
  memoizedProps: any,
  // 存储state更新的队列, 当前节点的state改动之后, 都会创建一个update对象添加到这个队列中.
  updateQueue: mixed,
  // 用于输出的state, 最终渲染所使用的state
  memoizedState: any,
  // 该fiber节点所依赖的(contexts, events)等
  dependencies: Dependencies | null,
  // 继承至父节点,影响本fiber节点及其子树中所有节点.
  // 与react应用的运行模式有关(有ConcurrentMode, BlockingMode, NoMode等选项).
  mode: TypeOfMode,
  // Effect 副作用相关
  flags: Flags, // 标志位
  subtreeFlags: Flags, //替代16.x版本中的 firstEffect, nextEffect. 当设置了enableNewReconciler=true才会启用
  deletions: Array<Fiber> | null, // 存储将要被删除的子节点. 当设置了enableNewReconciler=true才会启用
 // 单向链表, 指向下一个有副作用的fiber节点
  nextEffect: Fiber | null,
  // 指向副作用链表中的第一个fiber节点
  firstEffect: Fiber | null,
  // 指向副作用链表中的最后一个fiber节点
  lastEffect: Fiber | null,

```

```
  // 调度优先级相关
  lanes: Lanes, // 本fiber节点的优先级
  childLanes: Lanes, // 子节点的优先级

  // 指向内存中的另一个fiber,
  // 每个被更新过fiber节点在内存中都是成对出现(current和workInProgress)
  alternate: Fiber | null,
};
```


Fiber Tree
HostRootFIber
child
return
<App/>
child
return
div
child
return
return
return
header
sibling
<Content/>
footer
child
return
return
return
p
p
p

在2020年5月，调度优先级策略经历了比较大的重构。以 `expirationTime` 属性为代表的优先级模型被 `lane` 取代。详见这个PR

fiber.tag (workTag)

```
// 函数组件
export const FunctionComponent = 0;
// 类组件
export const ClassComponent = 1;
export const IndeterminateComponent = 2; // Before we know whether it is function or class
// HostRootFiber
export const HostRoot = 3; // Root of a host tree. Could be nested inside another node.
export const HostPortal = 4; // A subtree. Could be an entry point to a different renderer.
// HTML标签元素
export const HostComponent = 5;
export const HostText = 6;
export const Fragment = 7;
export const Mode = 8;
export const ContextConsumer = 9;
export const ContextProvider = 10;
export const ForwardRef = 11;
export const Profiler = 12;
export const SuspenseComponent = 13;
export const MemoComponent = 14;
export const SimpleMemoComponent = 15;
export const LazyComponent = 16;
export const IncompleteClassComponent = 17;
export const DehydratedFragment = 18;
export const SuspenseListComponent = 19;
export const FundamentalComponent = 20;
export const ScopeComponent = 21;
export const Block = 22;
export const OffscreenComponent = 23;
export const LegacyHiddenComponent = 24;
```

fiber.flags

```
// Don't change these two values. They're used by React Dev Tools.
export const NoFlags = /*                      */ 0b0000000000000000000;
```

```
3   export const PerformedWork = /*                    */ 0b0000000000000000001;
4
5   // You can change the rest (and add more).
6   export const Placement = /*                        */ 0b0000000000000000010;
7   export const Update = /*                           */ 0b0000000000000000100;
8   export const PlacementAndUpdate = /*               */ 0b0000000000000000110;
9   export const Deletion = /*                         */ 0b0000000000000001000;
10  export const ContentReset = /*                     */ 0b0000000000000010000;
11  export const Callback = /*                         */ 0b0000000000000100000;
12  export const DidCapture = /*                       */ 0b0000000000001000000;
13  export const Ref = /*                              */ 0b0000000000010000000;
14  export const Snapshot = /*                         */ 0b0000000000100000000;
15  export const Passive = /*                          */ 0b0000000001000000000;
16  // TODO (effects) Remove this bit once the new reconciler is synced to the
    old.
17  export const PassiveUnmountPendingDev = /*         */ 0b0000010000000000000;
18  export const Hydrating = /*                        */ 0b0000000010000000000;
19  export const HydratingAndUpdate = /*               */ 0b0000000010000000100;
20
21  // Passive & Update & Callback & Ref & Snapshot
22  export const LifecycleEffectMask = /*              */ 0b0000000001110100100;
23
24  // Union of all host effects
25  export const HostEffectMask = /*                   */ 0b0000000011111111111;
26
27  // These are not really side effects, but we still reuse this field.
28  export const Incomplete = /*                       */ 0b0000000100000000000;
29  export const ShouldCapture = /*                    */ 0b0000001000000000000;
30  export const ForceUpdateForLegacySuspense = /*     */ 0b0000100000000000000;
31
32  // Static tags describe aspects of a fiber that are not specific to a render,
33  // e.g. a fiber uses a passive effect (even if there are no updates on this
    particular render).
34  // This enables us to defer more work in the unmount case,
35  // since we can defer traversing the tree during layout to look for Passive
    effects,
36  // and instead rely on the static flag as a signal that there may be cleanup
    work.
37  export const PassiveStatic = /*                    */ 0b0010000000000000000;
38
39  // Union of side effect groupings as pertains to subtreeFlags
40  export const BeforeMutationMask = /*               */ 0b0000000001100001010;
41  export const MutationMask = /*                     */ 0b0000000010010011110;
42  export const LayoutMask = /*                       */ 0b0000000000010100100;
43  export const PassiveMask = /*                      */ 0b0000000001000001000;
44
45  // Union of tags that don't get reset on clones.
```

```
// This allows certain concepts to persist without recalculting them,
// e.g. whether a subtree contains passive effects or portals.
export const StaticMask = /*                    */ 0b0010000000000000000;

// These flags allow us to traverse to fibers that have effects on mount
// without traversing the entire tree after every commit for
// double invoking
export const MountLayoutDev = /*                */ 0b0100000000000000000;
export const MountPassiveDev = /*               */ 0b1000000000000000000;
```

双缓存

在 `React` 中最多会同时存在两棵 `Fiber树` 。当前屏幕上显示内容对应的 `Fiber树` 称为 `current Fiber树` ，正在内存中构建的 `Fiber树` 称为 `workInProgress Fiber树` 。他们通过 `alternate` 属性连接。

```
currentFiber.alternate === workInProgressFiber;
workInProgressFiber.alternate === currentFiber;
```

`React` 应用的根节点通过使 `current` 指针在不同 `Fiber树` 的 `rootFiber` 间切换来完成 `current Fiber` 树指向的切换。

即当 `workInProgress Fiber树` 构建完成交给 `Renderer` 渲染在页面上后，应用根节点的 `current` 指针指向 `workInProgress Fiber树` ，此时 `workInProgress Fiber树` 就变为 `current Fiber树` 。

每次状态更新都会产生新的 `workInProgress Fiber树` ，通过 `current` 与 `workInProgress` 的替换，完成 `DOM` 更新。